

Appendix H — Recommended Reading and Video Resources

Every entry below is cited because it left a real trace in the project — an ADR it justified, a library it got adopted or rejected, or a research-log entry it produced. This appendix is organized the way the book's own Research.md ledger is organized: not a syllabus, but a record of what was read before something was built, kept, or deliberately not built.

H.1 Foundational

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” NeurIPS 2020. The paper that names the pattern this entire book builds on and extends into an agentic, self-correcting form.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. “Attention Is All You Need.” NeurIPS 2017. Background for every transformer-based embedding and generation model this project depends on, from `all-MiniLM-L6-v2` to the chat models behind every LLM role in Appendix N.

H.2 Compression and Context Management

LangChain's `LLMChainExtractor`-style contextual-compression retriever documentation — evaluated as a candidate for the project's own context-compression stage and rejected: Decisions.md records it as adding extra latency and coupling compression to LangChain's own retriever format, which motivated the project's own hand-written NAC/DC/LBC pipeline (Chapters 22B–22C) instead of reusing this off-the-shelf component.

H.3 Self-Improvement and Weight-Level Adaptation

Zweiger, A., Pari, J., Guo, S., Akyürek, E., Kim, Y., and Agrawal, P. “Self-Adapting Language Models” (SEAL). NeurIPS 2025. arXiv:2506.10943. Read in full (Research.md topic 47) and then traced at the reference-implementation level across its ARC and SQuAD pipelines (topic 49) to understand how a model-generated “self-edit” is scored and distilled back into a policy — the clearest articulation found of what this project's own memory-injection loop does **not** yet do (Chapter 52.6).

“Continual Harness: Online Adaptation for Self-Improving Foundation Agents.” Read for its framing of an agent's harness — prompt, memory, skills, sub-agents — as something a second “Refiner” role can revise automatically while the agent works (Research.md topic 45), then traced at the code level against its reference implementation, the PokeAgent Challenge harness (topic 48). Filed as a candidate future direction for `self\_learner.py`; no implementation exists yet.

Akyürek, E. et al. Deductive Closure Training (2024). Cited alongside SEAL as adjacent self-improvement literature for Chapter 52's survey of where memory injection stops and weight-level self-adaptation begins.

H.4 Process Reward Models for Agents

“ToolPRMBench: A Benchmark for Process Reward Models in Tool-Using Agents.” Read May 2025 (Research.md topic 15) to evaluate whether a trained Process Reward Model should replace `check\_answer\_quality`'s hand-coded heuristic. Findings — general math-trained PRMs barely beat

random on tool-use, while a tool-specialized, RL-trained PRM (ToolPRM-GRPO) reached 78.6% average accuracy, beating GPT-5 as an LLM-judge — directly justified the project’s near-term choice (LLM-as-a-judge) and its longer-term aspiration (a domain-specific trained PRM, once enough thumbdown/interaction data accumulates).

H.5 Structured Decoding

Outlines — finite-state-machine constrained decoding library. Evaluated and rejected for this project’s Groq-backed pipeline (Research.md topic 6): FSM token masking only works against local model weights, so on a remote OpenAI-compatible API it silently degrades to the same ``response_format: json_schema`` passthrough the provider already offers natively.

Guidance — logit-masking constrained generation library. Rejected for the same structural reason as Outlines: no masking is possible against a remote inference API, so ``gen()`/`select()`` degrade to prompt engineering plus post-hoc parsing.

``json_repair`` (``pip install json-repair>=0.30``). Adopted as Tier 2 of ``fix_llm_output.py``’s repair pipeline (Research.md topic 17) — closes brackets, quotes unquoted keys, and fixes trailing commas without an LLM call, with the confirmed caveat that it is syntactic-only and its output must still be Pydantic-validated before use.

H.6 Framework Documentation

LangGraph — the state-machine/graph orchestration framework behind the entire ``app_workflow/`` pipeline (Part V onward). Its own ``StateGraph``, ``Send``, and ``Command(goto=..., update=...)`` documentation was the primary reference for the project’s dual-track parallel-compression graph.

LangSmith — read both as product documentation and as a course-style video transcript covering its Project → Trace → Run model, monitoring, evaluation, and dataset/annotation tooling (Research.md topic 52) — the one entry in this appendix sourced from a video rather than written material. Used to benchmark the project’s own hand-rolled logging/evaluation stack against a managed LLMOps platform, not adopted wholesale.

Arize Phoenix — self-hosted OpenTelemetry-native tracing backend, adopted as the project’s primary observability tool (ADR-063) after the 14-metric comparison in Research.md topic 54. Its OTel/OpenInference layering (topic 53) was required reading before diagnosing why ``get_current_span()`` doesn’t behave the way generic OTel documentation would suggest (Bugs.md BUG-071).

Langfuse — adopted as a third, parallel tracing backend (ADR-066). Its callback-based (rather than ambient) LangChain integration is documented in Research.md topic 56 as a genuine architectural outlier among the three backends this project evaluated, and its Scores/Datasets/Annotations model is covered in topics 57–59.

OpenInference Semantic Conventions — the standardized span-attribute vocabulary that lets Phoenix (and any other OTel-compatible backend) render AI-specific concepts — prompts, tokens, tool calls, retrieval results — consistently. Read as the third layer of the OTel → OpenInference → semantic-conventions stack (Research.md topic 53).